Dev Ready

AI 驱动的开发环境自助平台

面向开发人员的内部开发者平台（IDP）

k8s-ctl / 项目演讲 / 2026.11

开发环境准备中的重复工作

开发人员想要的

一个独立、可连接的、可快速访问验证的 Redis

实际需要处理的

镜像与版本　　配置与存储
访问入口　　端口占用记录
申请网络权限与资源
联系运维人员进行部署

面向开发人员的自助入口

“在我的 Namespace
部署一个 Redis。”

AI 客户端理解需求，平台执行标准部署

开发人员使用环境，管理员维护服务标准

一次自助申请的完整流程

01	表达需求	明确服务类型与 Namespace
02	选择标准	读取目录，补齐参数，生成 requestId
03	执行部署	保存配置，领用端口，预检查并提交
04	验证使用	查询就绪状态，验证协议与客户端连接

AI 与平台各自承担什么

AI 客户端

理解自然语言
补齐必要信息
编排工具并解释结果

平台后端

执行固定版本模板
记录配置与端口归属
回读状态并验证连接

模型在客户端运行，Kubernetes 操作在服务端执行

轻量的技术实现

交互入口	AI 客户端 / REST 调用方
接口与业务	MCP + HTTP / 标准部署协调器
持久记录	SQLite / 模板、修订、部署、端口与事件
集群执行	Kubernetes / client-go、kubectl、Helm

首批标准服务目录

数据库与中间件

Redis
PostgreSQL / MySQL
MongoDB / Milvus

应用与临时环境

Nginx
Python JupyterLab
沙箱 TTL 到期回收

7 类模板与流程已实现，真实集群验收需逐项完成

共享入口与端口台账

201

个端口的配置范围

30000–30200，含两端

共用一个 Higress LB
按实例分配 TCP 端口
同步已有占用，确认后释放

可用数量以实时台账为准

真实案例：Redis 访问验证

PONG

客户端认证与 Redis 响应验证通过

历史测试：wx-test-ns-002 / Higress TCP 30109

Operator CR 通过 kubectl 创建；实例、端口和卷现已清理

可重试、可查询、可回收

失败重试	复用 requestId，恢复原部署记录
配置追溯	固定修订、部署 YAML 与摘要
资源回收	核对 UID 与路由，确认端口可释放
数据与临时环境	数据库卷默认保留，沙箱按 TTL 清理

PoC 环境的风险控制与恢复

降低影响的部署要求

专用开发集群，与生产隔离
只使用测试数据，禁用真实敏感数据
限制操作权限与资源用量

操作失败后的恢复机制

相同 requestId 重试原流程
配置修订与部署记录可追溯
中断保留端口，确认后再释放

开发环境降低业务影响；集群级故障仍需管理员处理

当前阶段与下一步

已实现的 PoC 能力

REST 与 MCP 接入
标准配置与自动分配端口
状态查询、协议验证与回收

下一阶段

身份、归属授权与配额
Operator 生命周期适配
逐项真实验收与使用度量

当前用于受控开发环境，细粒度权限仍未实现

让开发人员更快开始开发

说出需求，
获得可用的开发环境

AI 理解需求，平台交付标准环境

试用关注：首次可用耗时、人工介入、成功率与回收情况

附录：现场演示顺序

演示前	确认版本、RBAC、镜像、存储和可用端口
申请时	读取目录，明确 Namespace，使用新 requestId
验证时	检查就绪状态，再验证客户端实际连接
结束时	回收实例并同步台账，数据卷按意图处理

附录：评委可能关心的问题

AI 在哪里？	模型在 MCP 客户端，服务端执行标准工具
与脚本的区别？	保存实例、修订、端口事件与失败恢复记录
多租户是否安全？	细粒度权限尚未完成，当前限定受控 PoC
为什么使用 SQLite？	减少部署依赖，当前采用单副本协调器

感谢聆听

Dev Ready

欢迎交流与提问

k8s-ctl